NAMA : THOMAS NONE LABINA

NIM / KELAS : 257111064 / C

MATA KULIAH : STRUKTUR DATA

```
class Mahasiswa:
    def __init__(self, nim, nama, jurusan):
        self.nim = nim
        self.nama = nama
        self.jurusan = jurusan

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None # Hapus baris ini kalau mau jadi Single Linked List

class LinkedListMahasiswa:
    def __init__(self, double=True):
        self.head = None
        self.tail = None
        self.double = double # True = Double, False = Single

    # 1. TAMBAH DI AWAL
    def tambah_awal(self, nim, nama, jurusan):
        mhs = Mahasiswa(nim, nama, jurusan)
        node = Node(mhs)
        if not self.head:
            self.head = self.tail = node
        else:
            node.next = self.head
            if self.double:
                self.head.prev = node
```

```
        self.head = node
    print(f"-> {nama} ditambah di AWAL")

# 2. TAMBAH DI AKHIR
def tambah_akhir(self, nim, nama, jurusan):
    mhs = Mahasiswa(nim, nama, jurusan)
    node = Node(mhs)
    if not self.head:
        self.head = self.tail = node
    else:
        self.tail.next = node
        if self.double:
            node.prev = self.tail
        self.tail = node
    print(f"-> {nama} ditambah di AKHIR")

# 3. TAMBAH DI TENGAH (URUT NIM)
def tambah_urut_nim(self, nim, nama, jurusan):
    mhs = Mahasiswa(nim, nama, jurusan)
    node = Node(mhs)

    if not self.head or nim < self.head.data.nim:
        self.tambah_awal(nim, nama, jurusan)
        return

    curr = self.head
    while curr.next and curr.next.data.nim < nim:
        curr = curr.next

    # Cek duplikat NIM
    if curr.data.nim == nim:
        print(f"!! NIM {nim} sudah ada!")
```

```
            return

        node.next = curr.next
        curr.next = node
        if self.double:
            if node.next:
                node.next.prev = node
            node.prev = curr
            if curr == self.tail:
                self.tail = node
        print(f"-> {nama} ditambah urut NIM")

    # 4. HAPUS BERDASARKAN NIM
    def hapus(self, nim):
        if not self.head:
            print("List kosong!")
            return

        curr = self.head
        while curr and curr.data.nim != nim:
            curr = curr.next

        if not curr:
            print(f"NIM {nim} tidak ditemukan!")
            return

        if curr == self.head:
            self.head = curr.next
            if self.head and self.double:
                self.head.prev = None
            if curr == self.tail:
                self.tail = None
```

```
        elif curr == self.tail:
            self.tail = curr.prev if self.double else self._get_prev(curr)
            if self.tail:
                self.tail.next = None
        else:
            prev_node = curr.prev if self.double else self._get_prev(curr)
            prev_node.next = curr.next
            if self.double and curr.next:
                curr.next.prev = prev_node

        print(f"-> Data NIM {nim} berhasil dihapus")

    def _get_prev(self, node_target): # Helper khusus Single Linked List
        curr = self.head
        while curr and curr.next != node_target:
            curr = curr.next
        return curr

    # 5. CARI BERDASARKAN NIM (Linear Search)
    def cari(self, nim):
        curr = self.head
        posisi = 0
        while curr:
            if curr.data.nim == nim:
                print(f"DITEMUKAN di posisi {posisi}: {curr.data.nim} | {curr.data.nama} |
{curr.data.jurusan}")
                return curr.data
            curr = curr.next
            posisi += 1
        print(f"NIM {nim} tidak ditemukan")
        return None

    # 6. TAMPILKAN SEMUA
```

```
    def tampilkan(self):
        if not self.head:
            print("Belum ada data mahasiswa")
            return
        print("\n--- DAFTAR MAHASISWA ---")
        print(f"{'NIM':<10} {'Nama':<15} {'Jurusan'}")
        print("-" * 40)
        curr = self.head
        while curr:
            print(f"{curr.data.nim:<10} {curr.data.nama:<15} {curr.data.jurusan}")
            curr = curr.next
        print("-" * 40 + f" [Mode: {'DOUBLE' if self.double else 'SINGLE'} Linked List]\n")

# --- PROGRAM UTAMA (MENU) ---
if __name__ == "__main__":
    print("Pilih implementasi:")
    print("1. Single Linked List")
    print("2. Double Linked List")
    pilih = input("Pilih (1/2): ")
    is_double = True if pilih == "2" else False

    ll = LinkedListMahasiswa(double=is_double)

    # DATA AWAL OTOMATIS
    data_awal = [
        (257111064, "Thomas", "Informatika"),
        (257111067, "Nogan", "Sistem Informasi"),
        (257111070, "Patrik", "Teknik Komputer"),
        (257111055, "Pedro", "Ilmu Komputer")

    ]
    for nim, nama, jurusan in data_awal:
```

```
    ll.tambah_urut_nim(nim, nama, jurusan)

print("\n[3 Data awal berhasil ditambahkan, silahkan pilih menu 6 untuk tampilkan]")

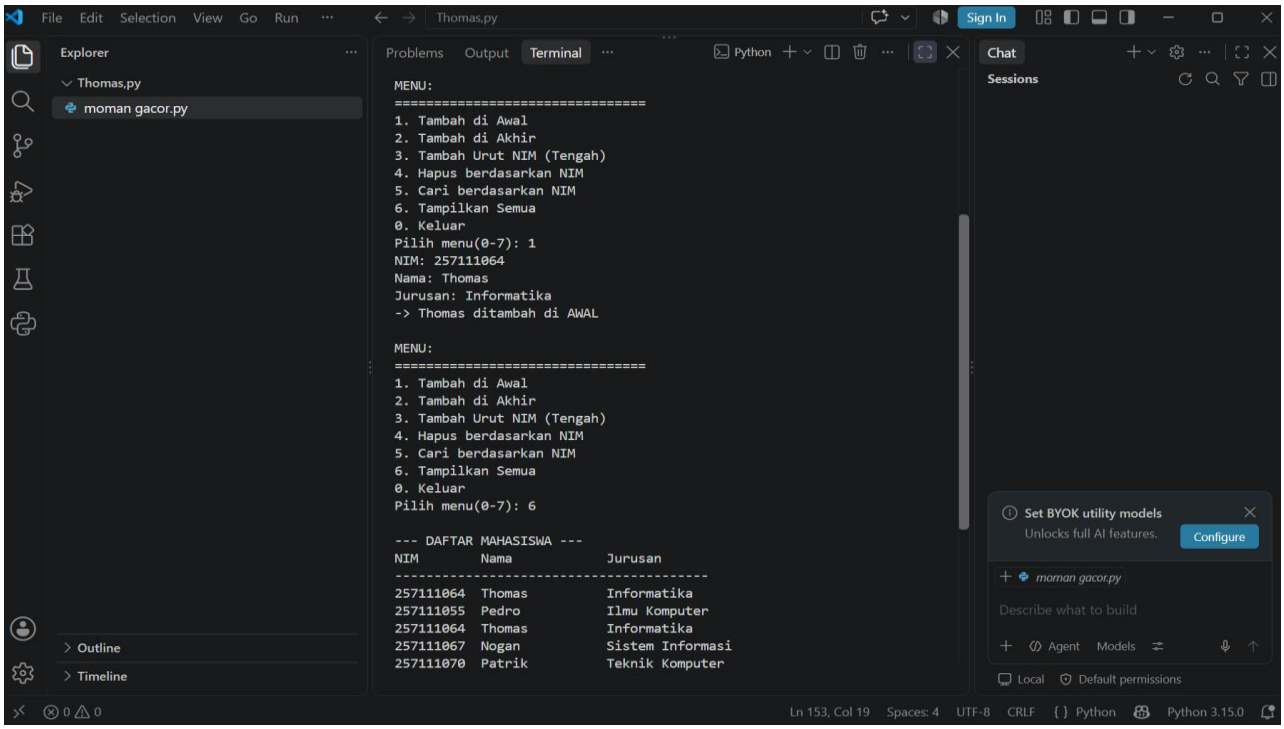
while True:
    print("\nMENU:")
    print("==================================")
    print("1. Tambah di Awal")
    print("2. Tambah di Akhir")
    print("3. Tambah Urut NIM (Tengah)")
    print("4. Hapus berdasarkan NIM")
    print("5. Cari berdasarkan NIM")
    print("6. Tampilkan Semua")
    print("0. Keluar")

    menu = input("Pilih menu(0-7): ")

    if menu == "0":
        break
    elif menu in ["1","2","3"]:
        nim = int(input("NIM: "))
        nama = input("Nama: ")
        jurusan = input("Jurusan: ")
        if menu == "1": ll.tambah_awal(nim, nama, jurusan)
        elif menu == "2": ll.tambah_akhir(nim, nama, jurusan)
        else: ll.tambah_urut_nim(nim, nama, jurusan)
    elif menu == "4":
        nim = int(input("NIM yang mau dihapus: "))
        ll.hapus(nim)
    elif menu == "5":
        nim = int(input("NIM yang dicari: "))
        ll.cari(nim)
```

```
elif menu == "6":
    ll.tampilkan()
```

HASILNYA



Analisis Kompleksitas Double Linked List

1. Analisis kompleksitas waktu (Time Complexity)

Kompleksitas waktu menunjukkan seberapa besar waktu yang dibutuhkan program ketika jumlah data mahasiswa bertambah. Simbol \(n\) menyatakan jumlah mahasiswa dalam linked list.

Operasi	Metode program	Kompleksitas waktu	Penjelasan
Tambah di awal	tambah_awal()	$O(1)$	Node baru langsung dihubungkan ke head.
Tambah di akhir	tambah_akhir()	$O(1)$	Pointer tail memungkinkan penambahan langsung di akhir.
Tambah berdasarkan urutan NIM	tambah_urut_nim()	$O(n)$	Program menelusuri node untuk menemukan posisi yang sesuai.
Hapus berdasarkan NIM	hapus()	$O(n)$	Program harus mencari NIM yang akan dihapus terlebih dahulu.
Cari berdasarkan NIM	cari()	$O(n)$	Pencarian dilakukan satu per satu dari awal list.
Tampilkan semua data	tampilkan()	$O(n)$	Semua node harus dikunjungi untuk menampilkan data.
Mencari node sebelumnya pada mode Single	_get_prev()	$O(n)$	Program menelusuri list dari awal untuk menemukan node sebelumnya.

Keterangan:

- \(O(1)\): waktu konstan, tidak bergantung pada jumlah data.
- \(O(n)\): waktu bertambah sebanding dengan jumlah data.

2. Analisis setiap operasi

A. Tambah di awal — \(O(1)\)

Pada metode `tambah_awal()`, program membuat node baru kemudian menghubungkannya ke node pertama.

Proses tersebut hanya membutuhkan beberapa perubahan pointer. Jumlah mahasiswa tidak memengaruhi banyaknya langkah utama.

B. Tambah di akhir — \(O(1)\)

Metode tambah_akhir() menggunakan pointer tail untuk mengakses node terakhir secara langsung.

Karena node terakhir dapat diakses tanpa menelusuri seluruh list, kompleksitasnya adalah \(O(1)\).

C. Tambah berdasarkan urutan NIM — \(O(n)\)

Metode tambah_urut_nim() menelusuri node menggunakan perulangan:

Dalam kasus terburuk, program perlu memeriksa hampir seluruh node sebelum menemukan posisi yang tepat. Karena itu, kompleksitasnya \(O(n)\).

Catatan: Pada kode saat ini, pengecekan NIM duplikat hanya dilakukan terhadap curr.data.nim. Jika NIM yang sama berada pada node berikutnya atau menjadi NIM terkecil di awal list, duplikat masih dapat masuk. Ini perlu diperbaiki agar validasi NIM berjalan dengan benar.

D. Hapus berdasarkan NIM — \(O(n)\)

Metode hapus() harus mencari node yang memiliki NIM sesuai sebelum menghapusnya.

Pencarian tersebut membutuhkan \(O(n)\) pada kasus terburuk. Setelah node ditemukan, pengubahan pointer untuk menghapus node hanya membutuhkan \(O(1)\).

Jadi, kompleksitas keseluruhannya adalah \(O(n)\).

E. Cari berdasarkan NIM — \(O(n)\)

Metode cari() menggunakan Linear Search, yaitu memeriksa data mahasiswa satu per satu.

- Kasus terbaik: \(O(1)\), jika data ditemukan pada node pertama.
- Kasus rata-rata: \(O(n)\).
- Kasus terburuk: \(O(n)\), jika data berada di akhir atau tidak ditemukan.

F. Tampilkan semua data — \(O(n)\)

- Metode `tampilkan()` mengunjungi setiap node untuk mencetak NIM, nama, dan jurusan. Jika terdapat 100 mahasiswa, program perlu memproses 100 node.
- Dengan demikian, kompleksitasnya adalah \(O(n)\).

3. Analisis kompleksitas ruang (Space Complexity)

Kompleksitas ruang menunjukkan jumlah memori yang dibutuhkan program.

Komponen	Kompleksitas ruang
Setiap node mahasiswa	$O(1)$
Seluruh linked list berisi \(n\) mahasiswa	$O(n)$
Variabel sementara seperti curr dan node	$O(1)$
Total struktur data	$O(n)$

Setiap mahasiswa disimpan dalam satu node yang memiliki data mahasiswa serta pointer next dan prev. Karena setiap penambahan mahasiswa menambah satu node, kebutuhan memori total bertumbuh secara linear.

4. Perbandingan Single dan Double Linked List

Aspek	Single Linked List	Double Linked List
Pointer setiap node	next	next dan prev
Tambah di awal	$O(1)$	$O(1)$
Tambah di akhir dengan tail	$O(1)$	$O(1)$
Pencarian data	$O(n)$	$O(n)$
Penghapusan berdasarkan NIM	$O(n)$	$O(n)$
Penghapusan node yang sudah diketahui	Perlu mengetahui node sebelumnya	$O(1)$
Kebutuhan memori	Lebih kecil	Lebih besar karena ada pointer tambahan

Perlu diperhatikan bahwa pada program ini, penghapusan berdasarkan NIM tetap \(O(n)\) pada Double Linked List karena program harus mencari NIM terlebih dahulu. Keuntungan Double Linked List adalah setelah node ditemukan, program dapat menggunakan prev untuk menghubungkan kembali node tetangganya tanpa menelusuri list dari awal.

5. Kesimpulan untuk laporan

Double Linked List pada program pengelolaan data mahasiswa memiliki kompleksitas waktu \(O(1)\) untuk penambahan di awal dan akhir karena program menggunakan pointer head dan tail. Sementara itu, penambahan berdasarkan urutan NIM, penghapusan berdasarkan NIM, pencarian, dan penampilan seluruh data memiliki kompleksitas waktu \(O(n)\) karena melibatkan penelusuran node. Kompleksitas ruang keseluruhan adalah \(O(n)\), sebab setiap mahasiswa membutuhkan satu node dalam linked list.

Dengan demikian, Double Linked List cocok untuk pengelolaan data yang membutuhkan fleksibilitas dalam penyisipan dan penghapusan node, terutama ketika node yang akan dihapus sudah diketahui. Namun, untuk pencarian berdasarkan NIM, program ini masih membutuhkan waktu linear karena belum menggunakan struktur pencarian tambahan seperti hash table atau balanced search tree.